

SISTEMAS INTELIGENTES

APRENDIZAJE DEL USO DE LAS REDES NEURONALES



UNIVERSIDAD DE MÁLAGA

Iván Calvo Santos MIM

74541138R

Índice

1. RED DE HOPFIELD -----	3
1.a. OBJETIVOS -----	3
1.b. ENTRENAMIENTO DE LA RED -----	4
1.c. DISTORSIÓN DEL PATRÓN -----	5
1.d. APLICACIÓN DE HOPFIELD -----	6
1.e. RESULTADOS -----	7
2. REDES NEURONALES COMPETITIVAS -----	8
2.a. OBJETIVOS -----	8
2.b. ENTRENAMIENTO DE LA RED -----	9
2.c. COMPRIMIR Y MOSTRAR PATRONES -----	10
2.d. RESULTADOS -----	11
3. PERCEPTRÓN MULTICAPA -----	13
3.a. OBJETIVOS -----	13
3.b. ENTRENAMIENTOD DE LA RED -----	14
3.c. RESULTADOS -----	17
4. RED NEURONAL CONVOLUCIONAL -----	22
4.a. OBJETIVOS -----	22
4.b. ENTRENAMIENTOD DE LA RED -----	23
4.c. RESULTADOS -----	25
5. TRATAMIENTO DE DATOS -----	28
5.a. OBJETIVOS -----	28
5.b. DATOS NULOS, DUPLICADOS Y ELIMINAR COLUMNAS -----	29
5.c. TRATAR TEXTOS -----	31
5.d. TRATAR OUTLIERS -----	34
5.e. PERCEPTRÓN MULTICAPA -----	35

1. REDES DE HOPFIELD

a. OBJETIVOS

Dados ciertas imágenes de números y puntos, se entrena una Red de Hopfield con el fin de recordar dichos patrones. Estas imágenes se distorsionarán con el fin de que la Red de Hopfield entrenada rehaga la imagen.

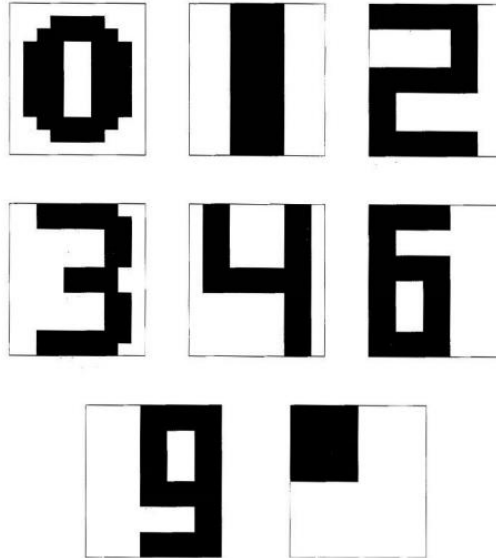


Ilustración 1.1: Patrones a Recordar

b. ENTRENAMIENTO DE LA RED

Dados los patrones se utilizan 2 funciones creadas con el fin de crear la matriz de pesos de las neuronas de la Red de Hopfield.

La primera de ellas obtiene el array tridimensional que contiene todos los patrones bidimensionales. Cada patrón bidimensional se convertirá en un vector utilizando la función creada “matriz_a_vector()”.

```
def crear_pesos_hopfield(patrones):  
    num_patrones = patrones.shape[0]  
    num_elementos = patrones[0].size  
    W = np.zeros((num_elementos, num_elementos))  
  
    # Recorremos todos los patrones  
    for i in range(num_patrones):  
        patron_matriz = patrones[i]  
        v = matriz_a_vector(patron_matriz)  
        W = actualizar_pesos(v, W)  
  
    return W
```

Ilustración 2.2: Función “crear_pesos_hopfield()”

Estos vectores serán utilizados para actualizar la matriz de pesos utilizando la siguiente función. Esta función multiplica el vector por su traspuesto, suma la matriz resultante a la matriz de pesos anterior, y elimina la diagonal con el fin de actualizar la matriz de pesos.

```
def actualizar_pesos(vector, W_actual):  
    matriz_patron = np.outer(vector, vector)  
    W_nueva = W_actual + matriz_patron  
    np.fill_diagonal(W_nueva, 0)  
    return W_nueva
```

Ilustración 3.3: Función “actualizar_pesos()”

c. DISTORSIÓN DEL PATRÓN

Dada el patrón a distorsionar (matriz), se realiza una copia para que el patrón original no se distorsione (con el fin de observar la diferencia al final del uso de la Red de Hopfield). Cada “n” pixeles de cada una de las filas del patrón se invertirá de forma aleatoria.

```
def distorsionar_patron(matriz, n):  
  
    matriz = np.array(matriz, copy=True)  
  
    filas, columnas = matriz.shape  
  
    for i in range(filas):  
        for _ in range(n):  
            # Número aleatorio entre 0 y columnas-1  
            idx = np.random.randint(0, columnas)  
  
            # Cambiar signo del elemento seleccionado  
            matriz[i, idx] *= -1  
  
    return matriz
```

Ilustración 4.4: Función “distorsionar_patron()”

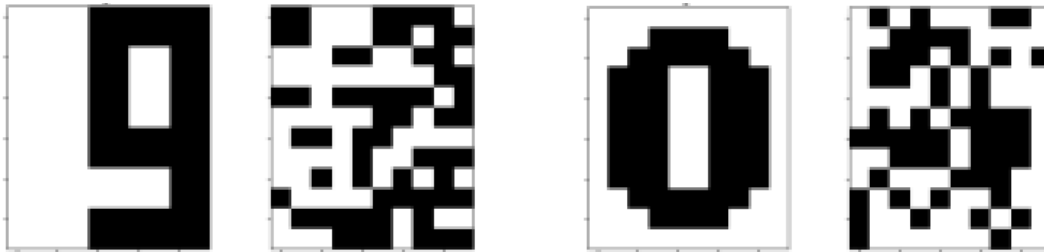


Ilustración 5.5: Ejemplo de Patrones Distorsionados

d. APLICACIÓN DE LA RED DE HOPFIELD

Una vez obtenido el patrón distorsionado, solo queda aplicar la Red de Hopfield con el fin de observar si consigue retornar el patrón inicial.

```
def aplicar_hopfield(W, patron, i_max):  
  
    filas, columnas = patron.shape  
  
    # Pasamos a vector  
    v = matriz_a_vector(patron)  
  
    iteracion = 0  
    fin = False  
  
    while not fin:  
        # Multiplicación W * v  
        v_nuevo = W @ v  
  
        # Cada elemento se cambia por su signo  
        v_nuevo = np.sign(v_nuevo)  
  
        iteracion += 1  
  
        # Condiciones de parada:  
        # 1) se alcanza el máximo de iteraciones  
        # 2) el vector no cambia respecto al anterior  
        if iteracion >= i_max or np.array_equal(v_nuevo, v):  
            fin = True  
  
        # Actualizamos el vector para la siguiente iteración  
        v = v_nuevo  
  
    # Volvemos a forma de matriz  
    matriz_final = v.reshape(filas, columnas)  
  
    return matriz_final
```

Ilustración 6.6: Función “aplicar_hopfield()”

Esta función utilizará un patrón y lo multiplicará por la matriz de pesos cambiando los valores del patrón. Estos valores se pueden encontrar fuera del rango $[-1, 1]$, por lo que se utilizará la función “sign()” para dejarlos dentro de dicho rango.

La Red de Hopfield, al ser entrenada con varios patrones puede no llegar a recordar bien un patrón dada cierta imagen distorsionada. Es por eso que se pondrá una condición de no sobrepasar un máximo de iteraciones y otra de no modificación del patrón tras una iteración. Si se cumple alguna de esas dos iteraciones, la aplicación terminará.

e. RESULTADOS

Como se ha nombrado al principio, esta Red de Hopfield entrenada es capaz de recuperar sin problema todos los patrones con los cuales ha sido entrenada.

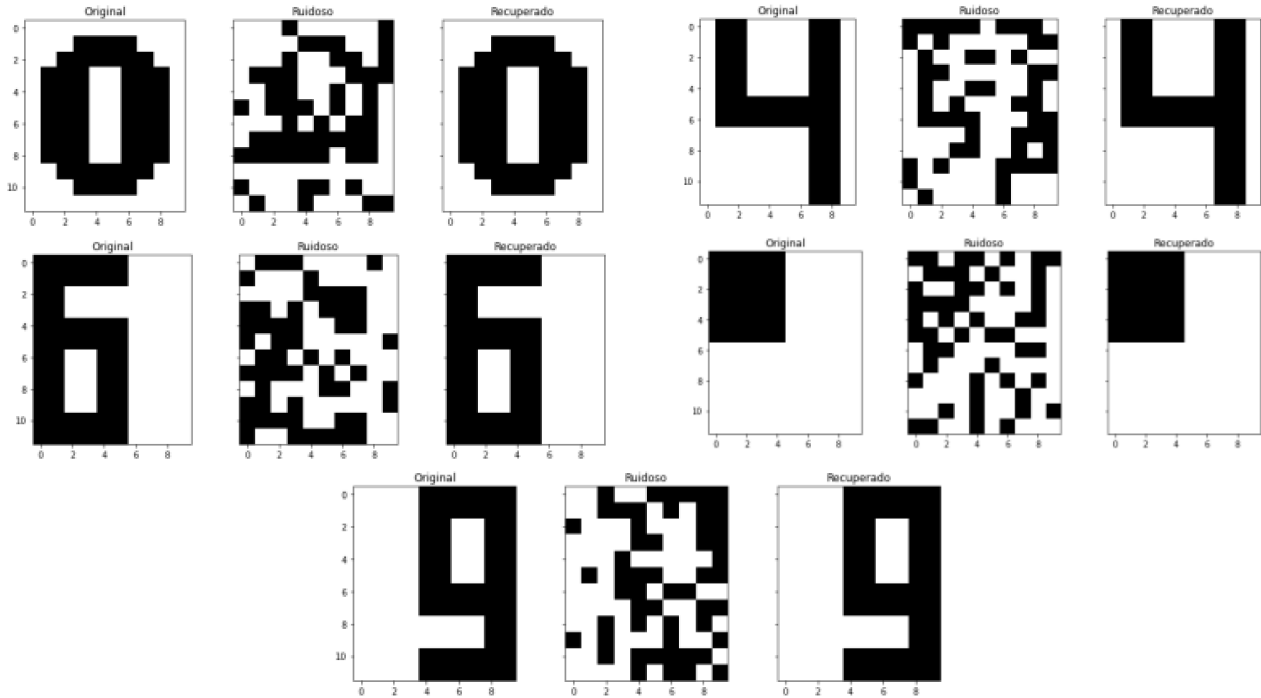


Ilustración 7.7: Resultados Correctos

Sin embargo, como se comentó en el apartado anterior, existen ciertas distorsiones que hacen que la red, al haber aprendido varios patrones, se establezca en patrones erróneos (e incluso otros patrones diferentes).

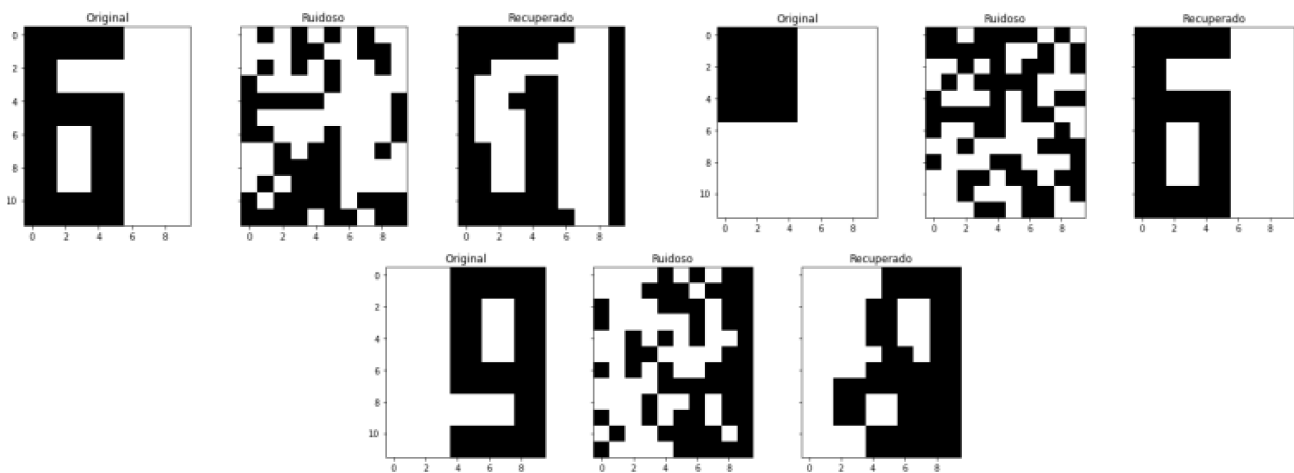


Ilustración 8.8: Resultados Erróneos

2. REDES NEURONALES COMPETITIVAS

a. OBJETIVOS

Dadas 2 imágenes (una en escala de grises y otra RGB) se pide comprimir la imagen en una de menos capacidad utilizando una red competitiva. Esta red sustituirá espacios de “ $n \times n$ ” píxeles por un patrón de los que haya aprendido.



Ilustración 2.1: Imágenes a Comprimir

Para ello la práctica nos da 2 funciones que deberemos emplear:

- “mayor_pot_sinaptico()”: Calcula el potencial sináptico de cada neurona y devuelve el índice de la neurona ganadora (la más cercana al patrón)
- “entrenamiento_competitivo()”: Entrena la red competitiva usando patrones iniciales aleatorios y recorre varias épocas de entrenamiento reduciendo la tasa de aprendizaje, seleccionando un patrón y ajustando sus pesos.

b. ENTRENAMIENTO DE LA RED COMPETITIVA

Para entrenar la red será necesario pasar cada subimagen de la imagen como un vector ordenado. Para ello se utilizará la función “imagen_a_patrones()” la cual dado el tamaño en pixeles de la subimagen, crea los vectores ordenados de todas las subimagenes colocándolos en una tabla.

```
def imagen_a_patrones(imagen, tam_cuadrado, flag_mostrar):  
    alto, ancho = imagen.shape  
  
    num_bloques_alto = alto // tam_cuadrado  
    num_bloques_ancho = ancho // tam_cuadrado  
  
    num_ventanas = num_bloques_alto * num_bloques_ancho  
  
    patrones = np.zeros((tam_cuadrado * tam_cuadrado, num_ventanas))  
  
    columna = 0  
    for i_bloque in range(num_bloques_alto):  
        for j_bloque in range(num_bloques_ancho):  
            # Coordenadas de la esquina superior izquierda de la ventana  
            fila_ini = i_bloque * tam_cuadrado  
            col_ini = j_bloque * tam_cuadrado  
  
            # Extraemos la ventana [fila_ini : fila_ini+tam, col_ini : col_ini+tam]  
            ventana = imagen[fila_ini:fila_ini + tam_cuadrado,  
                             col_ini:col_ini + tam_cuadrado]  
  
            # Aplanamos la ventana en un vector columna  
            # (el orden por defecto es por filas, que es lo habitual)  
            patrones[:, columna] = ventana.reshape(-1)  
  
            columna += 1
```

Ilustración 2.2: Fragmentos de la Función “imagen_a_patrones()”

Una vez creados los patrones, se entrena la red utilizando la función “entrenamiento_competitivo()” anteriormente nombrada.

```
patrones = imagen_a_patrones(img , tam_cuadrado, True)  
W = entrenamiento_competitivo(tasa_aprendizaje, n_epocas, n_sub_imagenes, patrones)
```

Ilustración 2.3: Entrenamiento de los Patrones

c. COMPRIMIR Y MOSTRAR PATRONES

Una vez entrenada la Red Competitiva, se procede a comprimir los patrones. Para ello se comprueba la neurona más parecida al patrón comprobando su potencial sináptico. Todo esto se realiza en la función “comprimir_patrones()”.

```
def comprimir_patrones(W, patrones):  
    """  
    Sustituye cada patrón por el prototipo (columna de W) de su neurona ganadora.  
    """  
    dim, num_patrones = patrones.shape  
    dimW, num_neuronas = W.shape  
  
    patrones_comp = np.zeros_like(patrones)  
  
    for j in range(num_patrones):  
        patron = patrones[:, j]  
        # neurona ganadora usando tu función  
        r = mayor_pot_sinaptico(W, patron, num_neuronas)  
        # copiamos el prototipo correspondiente  
        patrones_comp[:, j] = W[:, r]  
  
    return patrones_comp
```

Ilustración 2.5: Función “comprimir_patrones()”

Finalmente, una vez sustituidos los patrones reales por su versión comprimida (dado por su neurona ganadora), se deben convertir los nuevos patrones (vectores) en imágenes para su visualización. Para ello se utilizará la función “visualizar_patrones()” la cual convierte cada patrón en una subimagen y la coloca en su posición para completar la imagen total.

```
for i in range(num_bloques_alto):  
    for j in range(num_bloques_ancho):  
  
        ax = axes[i, j]  
        if patrones.ndim == 2:  
  
            vector = patrones[:, idx]  
            subimagen = vector.reshape((tam_cuadrado, tam_cuadrado))  
  
            ax.imshow(subimagen, cmap='gray', interpolation='nearest', vmin = 0.0, vmax = 1.0)  
  
        else:  
  
            patch = patrones[:, idx, :]  
            subimagen = patch.reshape((tam_cuadrado, tam_cuadrado, 3))  
  
            ax.imshow(subimagen, interpolation='nearest', vmin=0.0, vmax=1.0)  
  
        ax.axis('off')  
        idx += 1
```

Ilustración 2.6: Fragmentos de la Función “visualizar_patrones()”

d. RESULTADOS

Una vez explicadas las funciones se utilizarán con el fin de comprimir las 2 imágenes de la “Ilustración 2.1.” utilizando los siguientes parámetros:

- 5 Sub Imágenes de 16 x 16 con 5 épocas de entrenamiento

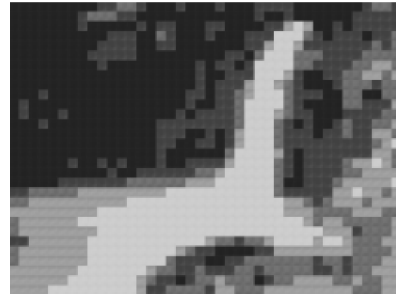
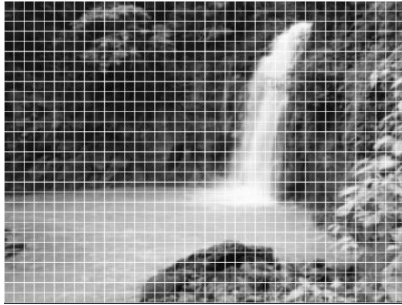


Ilustración 2.7: 5 Sub Imágenes de 16 x 16 con 5 Épocas

- 50 Sub Imágenes de 32 x 32 con 1000 épocas de entrenamiento

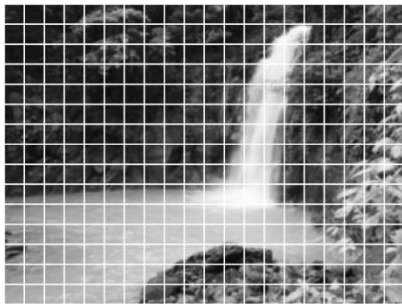


Ilustración 2.8: 50 Sub Imágenes de 32 x 32 con 1000 Épocas

- 100 Sub Imágenes de 16 x 16 con 100 épocas de entrenamiento

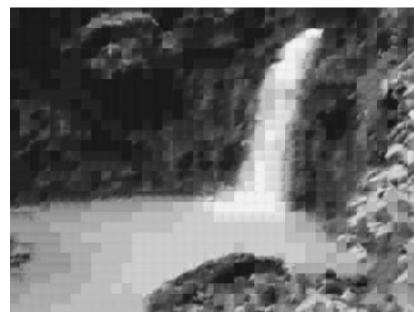
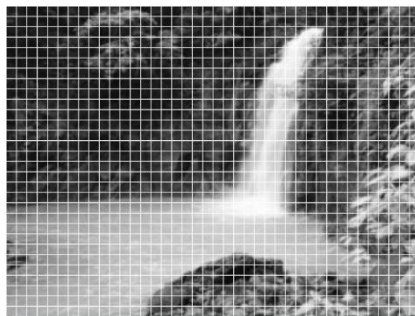


Ilustración 2.9: 100 Sub Imágenes de 16 x 16 con 100 Épocas

- 64 Sub Imágenes de 16 x 16 con 5 épocas: Para la imagen de color se repite el procedimiento para cada valor R, G y B.

```
pat_R = imagen_a_patrones(R, tam_cuadrado, False)
W_R = entrenamiento_competitivo(tasa_aprendizaje, n_epocas, n_sub_imagenes, pat_R)
comp_R = comprimir_patrones(W_R, pat_R)

pat_G = imagen_a_patrones(G, tam_cuadrado, False)
W_G = entrenamiento_competitivo(tasa_aprendizaje, n_epocas, n_sub_imagenes, pat_G)
comp_G = comprimir_patrones(W_G, pat_G)

pat_B = imagen_a_patrones(B, tam_cuadrado, False)
W_B = entrenamiento_competitivo(tasa_aprendizaje, n_epocas, n_sub_imagenes, pat_B)
comp_B = comprimir_patrones(W_B, pat_B)
```



Ilustración 2.10: 64 Sub Imágenes a Color de 16 x 16 con 5 Épocas

3. PERCEPTRÓN MULTICAPA

a. OBJETIVOS

Dados unos conjuntos de datos, se pide entrenar una red neuronal con el fin de realizar 2 ejercicios de clasificación y uno de regresión.

b. ENTRENAMIENTO DE LA RED

Para entrenar la red se utilizarán una de las siguientes funciones creadas (dependiendo de si es un problema de clasificación o regresión), las cuales tienen varios parámetros de entrada que el usuario podrá modificar con el fin de adaptar el entrenamiento de la red neuronal.

```
def Clasificacion_Multi(dataset = None,
                        X_train = None, Y_train = None, X_val = None, Y_val = None, X_test = None, Y_test = None,
                        porcentaje_train = 0.8, porcentaje_test = 0.0,
                        scaler = None, n_capa_oculta = 8, epocas = 200,
                        one_hot_encoding = False, patience = None):
```

```
def Regresion(dataset, porcentaje_train = 0.6, porcentaje_test = 0.2,
              scalerX = None, scalerY = None, learning_rate = None,
              n_capa_oculta = 16, epocas = 200, patience = None):
```

Ilustración 3.1: Funciones “Clasifiacion_Multi()” y “Regresion()”

Ambas funciones comenzarán separando los datos “X” e “Y” en 3 grupos cada uno. Uno de entrenamiento (train) con los cuales se entrenará la red neuronal, otro de validación (val) con los cuales se irá comprobando el entrenamiento durante su realización, y el último de prueba (test) con los cuales se comprobarán los resultados del entrenamiento.

```
X_train, Y_train, X_val, Y_val, X_test, Y_test = mc.split_all(dataset, porcentaje_train, porcentaje_test)
```

Ilustración 3.2: Uso de la Función “split_all()”

La función “split_all()” tiene en su interior el comando “train_test_split()” el cual realiza la división de datos.

A continuación se realiza el escalado si el usuario ha decidido escalar los datos. En el problema de clasificación solo se escalan los datos “X”, sin embargo, en los de regresión se pueden escalar también los datos “Y”.

```
if scaler is not None:
    X_escalado, X_train, X_val, X_test = mc.escalado(scaler, X, X_train, X_val, X_test)
```

Ilustración 3.3: Uso de la Función “escalado()”

La función “escalado()” tiene en su interior los comandos “scaler.fit_transform()” y “scaler.transform()” los cuales realizan el escalado.

El siguiente paso creará el modelo el cual tendrá 2 capas. El número de entradas del modelo será igual el número de inputs (número de columnas de la variable “X”) y el número de salidas al número de clases en problemas de clasificación, o una sola salida en problemas de regresión.

La función de activación de la capa de salida dependerá del problema. En cuanto a clasificación es recomendable “softmax” y en cuanto a regresión “relu” o “linear”.

```
modelo = keras.Sequential([
    keras.layers.Dense(n_capa_oculta, activation="softmax", input_shape=[X.shape[1]]),
    keras.layers.Dense(n_salida, activation="softmax")
])

modelo = keras.Sequential([
    keras.layers.Dense(n_capa_oculta, activation="relu", input_shape=[X.shape[1]]),
    keras.layers.Dense(len(dataset.target_names), activation="linear")
])
```

Ilustración 3.4: Modelos de Clasificación y Regresión Respectivamente.

Como penúltimo paso deberemos elegir la función de pérdida y el compilador requerido. En el caso de usar el método de clasificación, si elegimos utilizar “one_hot_encoding”, la función de pérdida será “categorical_crossentropy” y no “sparse_categorical_crossentropy”. En el caso de usar el método de regresión, la función de pérdida será “mean_squared_error”.

```
if one_hot_encoding:
    if Y_train is not None:
        Y_train = keras.utils.to_categorical(Y_train)
    if Y_val is not None:
        Y_val = keras.utils.to_categorical(Y_val)
    if Y_test is not None:
        Y_test = keras.utils.to_categorical(Y_test)

    modelo.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
else:
    modelo.compile(loss="sparse_categorical_crossentropy", optimizer="adam", metrics=["accuracy"])

# Accuracy solo cuenta el porcentaje de veces que y_pred = y_val
if learning_rate is not None:
    modelo.compile(loss="mean_squared_error", optimizer=keras.optimizers.Adam(learning_rate=learning_rate))
else:
    modelo.compile(loss="mean_squared_error", optimizer="adam") # metrics=[keras.metrics.MeanSquaredError()]
```

Ilustración 3.5: Compilación de los Modelos de Clasificación y Regresión.

Finalmente si queremos utilizar “Parada Temprana” con el fin de que no exista sobreajuste, se entrenará el modelo con o sin parada temprana.

```
if patience is not None:
    parada_temprana = keras.callbacks.EarlyStopping(patience=patience)
    resultados = modelo.fit(X_train,Y_train, epochs = epocas, validation_data=(X_val,Y_val), callbacks=[parada_temprana])
else:
    resultados = modelo.fit(X_train,Y_train, epochs = epocas, validation_data=(X_val,Y_val))
```

Ilustración 3.6: Elección de Parada Temprana

c. RESULTADOS

Con el fin de mostrar el funcionamiento de los Perceptrones Multicapa entrenados de la forma descrita en el apartado anterior, se utilizarán las funciones “Info_Clas_Mult()” y “Info_Reg()” las cuales grafican los resultados, reducen el número de dimensiones, eliminan el escalado, y realizan la matriz de confusión...

```
def Info_Clas_Mult ( dataset = None,
                    X_train = None, Y_train = None, X_val = None, Y_val = None, X_test = None, Y_test = None,
                    n_dim_PCA = None, scaler_PCA = None, scaler = None,
                    resultados = None, matriz_conf = False, datos_terminal = False,
                    Y_pred = None, one_hot_encoding = False):

def Info_Reg ( dataset = None,
               X_train = None, Y_train = None, X_val = None, Y_val = None, X_test = None, Y_test = None,
               resultados = None, histograma = False, datos_terminal = False,
               scalerX = None, scalerY = None,
               Y_pred = None):
```

Ilustración 3.7: Funciones “Info_Clas_Multi()” y “Info_Reg()”

A continuación se comentarán los 3 ejercicios deseados.

- Ejercicio 1 – Clasificación:

```
from sklearn.datasets import load_iris
dataset = load_iris()
modelo, resultados, X_train, Y_train, X_val, Y_val, X_test, Y_test, scaler = Clasificacion_Multi(dataset = dataset, one_hot_encoding = True,
                                                                                             patience = 1, epocas = 300)
Y_pred = modelo.predict(X_val)
Info_Clas_Mult( dataset = dataset,
                 X_train = X_train, Y_train = Y_train, X_val = X_val, Y_val = Y_val, X_test = X_test, Y_test = Y_test,
                 n_dim_PCA = 2, scaler = scaler,
                 resultados = resultados, matriz_conf = True, datos_terminal = True,
                 Y_pred = Y_pred, one_hot_encoding = True)
```

Ilustración 3.8: Ejecución del Ejercicio 1.

Tras 300 épocas de entrenamiento se obtiene la siguiente gráfica de evolución mostrando como la precisión aumenta mientras que la pérdida disminuye por cada época que pasa.

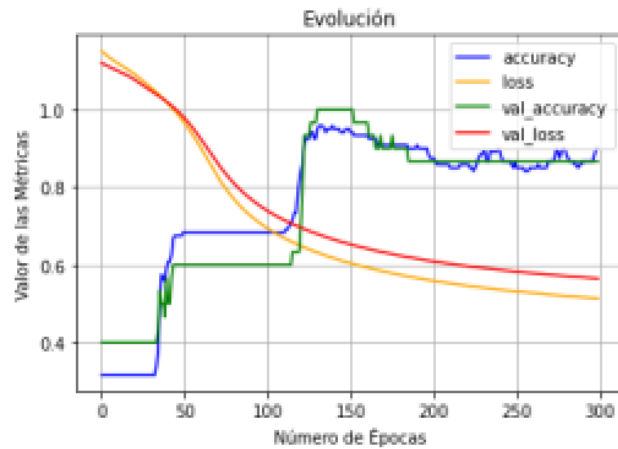


Ilustración 3.9: Evolución Ejercicio 1

Observando la matriz de confusión (utilizando los datos predichos por el modelo) podemos observar como el modelo casi siempre acierta. Sin embargo algún dato de la clase 1 es confundido por uno de la clase 2.

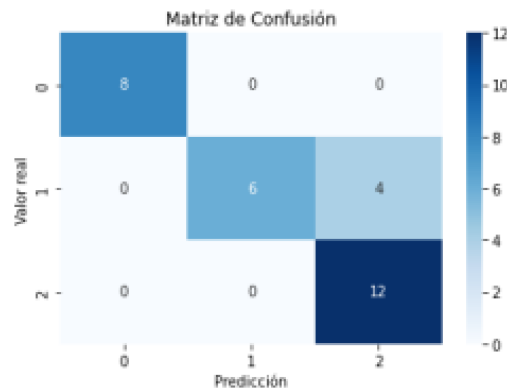


Ilustración 3.10: Matriz de Confusión Ejercicio 1.

- Ejercicio 2 – Regresión:

```
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler

dataset = fetch_california_housing()

modelo, resultados, X_train, Y_train, X_val, Y_val, X_test, Y_test, scalerX, scalerY = Regresion(dataset = dataset,
                                                    scalerX = StandardScaler(), scalerY = StandardScaler(),
                                                    patience = None, epocas = 200, learning_rate = 0.01)

Y_pred = modelo.predict(X_test)

Info_Reg(dataset = dataset,
          X_train = X_train, Y_train = Y_train, X_val = X_val, Y_val = Y_val, X_test = X_test, Y_test = Y_test,
          resultados = resultados, histograma = True, datos_terminal = True,
          scalerX = scalerX, scalerY = scalerY,
          Y_pred = Y_pred)
```

Ilustración 3.11: Ejecución del Ejercicio 2.

Tras 200 épocas de entrenamiento se observa en la “Ilustración 3.12.” como por cada época entrenada la pérdida del modelo entrenado se reduce.

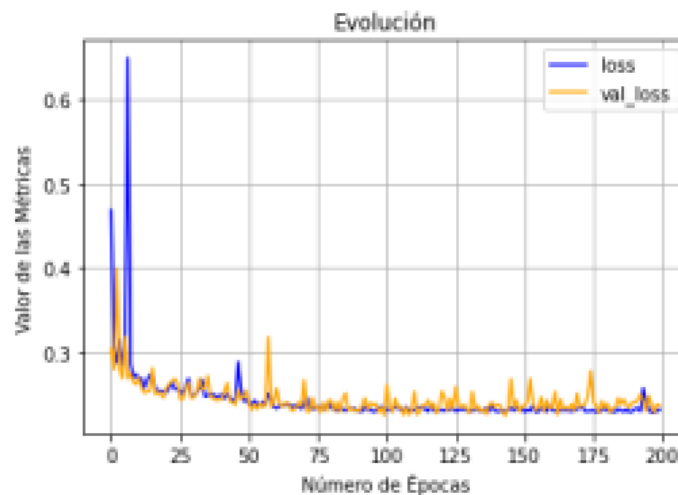


Ilustración 3.12: Evolución Ejercicio 2.

Tras entrenar el modelo, este da un error medio de “0.406”, es decir, de aproximadamente “406000\$”. No se ha conseguido reducir dicho valor.

Error Medio
0.4062045124261774

Ilustración 3.13: Error Medio Ejercicio 2.

- Ejercicio 3 – Clasificación:

```
mnist = keras.datasets.mnist
(X_train, Y_train), (X_test, Y_test) = mnist.load_data()
X_train = mc.imagen2linea(X_train)
X_test = mc.imagen2linea(X_test)

mc.informacion_var(X_train)
mc.informacion_var(Y_train)
mc.informacion_var(X_test)
mc.informacion_var(Y_test)

# No sé con cual está mejor, si con "one_hot" o sin.
modelo, resultados, X_train, Y_train, X_val, Y_val, X_test, Y_test, scaler = Clasificacion_Multi(X_train = X_train, Y_train = Y_train, X_val = X_test, Y_val = Y_test, X_test = X_test, Y_test = Y_test,
epocas = 20, one_hot_encoding = True, scaler = MinMaxScaler())

Y_pred = modelo.predict(X_val)
Info_Clas_Mult( X_train = X_train, Y_train = Y_train, X_val = X_test, Y_val = Y_test, X_test = X_test, Y_test = Y_test,
scaler = scaler,
resultados = resultados, matriz_conf = True, datos_terminal = True,
Y_pred = Y_pred, one_hot_encoding = True)
```

Ilustración 3.14: Ejecución del Ejercicio 3.

En este ejercicio se tratan imágenes las cuales no pueden ser entradas de un Perceptrón Multicapa. Debido a ello, al igual que en la práctica anterior, pasamos la imagen a un patrón (vector) el cual será introducido a la red neuronal.

Tras 20 épocas se muestra como por cada época realizada la precisión del modelo aumenta y la pérdida disminuye.

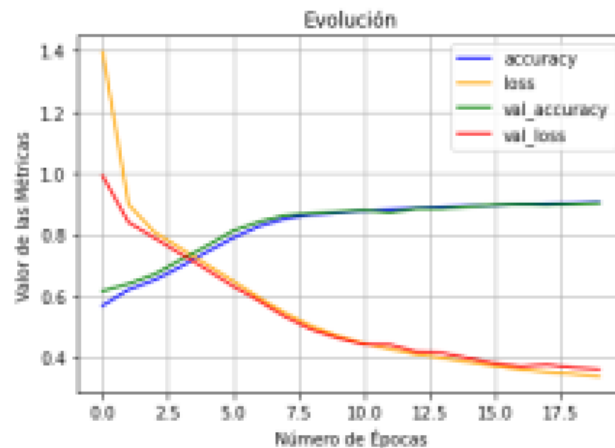


Ilustración 3.15: Evolución Ejercicio 3.

Finalmente se puede comprobar en la matriz de confusión que el modelo acierta la mayoría de las veces el resultado real.

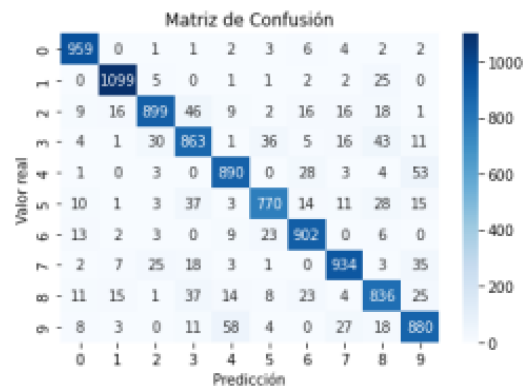


Ilustración 3.16: Matriz de Confusión Ejercicio 3.

4. RED NEURONAL CONVOLUCIONAL

a. OBJETIVOS

Dados unos conjuntos de datos representados por imágenes, se pide entrenar una red neuronal convolucional con el fin de realizar 2 ejercicios de clasificación.

b. ENTRENAMIENTO DE LA RED

Al igual que en la práctica anterior, para entrenar la red se utilizará la siguiente función creada, la cual tienen varios parámetros de entrada que el usuario podrá modificar con el fin de adaptar el entrenamiento de la red neuronal convolucional.

```
def CNN(dataset = None,
        X_train = None, Y_train = None, X_val = None, Y_val = None, X_test = None, Y_test = None,
        porcentaje_train = 0.8, porcentaje_test = 0.0,
        scaler = False,
        n_filtros = 64, tam_filtro = 4, tam_pool = 2, dropout = 0.2, n_capas_CNN = 1, batch_dropout = False,
        n_capa_oculta = 50, epocas = 10,
        one_hot_encoding = False, patience = None):
```

Ilustración 4.1: Función CNN()

Esta función es muy parecida a las utilizadas en la práctica anterior, por lo que varias imágenes no se mostrarán.

Tras separar los datos, el escalado (al tratarse de imágenes) se consigue dividiendo cada pixel entre 255 para que se encuentren en el intervalo [0, 1] en el cual trabajan mejor.

El modelo a crear también será parecido, sin embargo se añadirán capas convolucionales en las que los tamaños tanto de las capas de “Pool”, como de los filtros serán elegidos por el usuario. El usuario también podrá elegir qué cantidad de capas tendrá la Red Neuronal Convolucional y si tendrá capas de “Batch” o “Dropout”).

```
modelo = None
if n_capas_CNN == 1:
    if batch_dropout:
        modelo = keras.Sequential([
            keras.layers.Conv2D(filters = n_filtros, kernel_size=tam_filtro, activation = "relu", input_shape=[tamX,tamY,RGB]),
            keras.layers.MaxPool2D(pool_size=tam_pool),
            keras.layers.BatchNormalization(),
            keras.layers.Flatten(),
            keras.layers.Dropout(dropout),
            keras.layers.Dense(n_capa_oculta, activation="relu"),
            keras.layers.Dense(n_salida, activation="softmax")
        ])
    else:
        modelo = keras.Sequential([
            keras.layers.Conv2D(filters = n_filtros, kernel_size=tam_filtro, activation = "relu", input_shape=[tamX,tamY,RGB]),
            keras.layers.MaxPool2D(pool_size=tam_pool),
            keras.layers.Flatten(),
            keras.layers.Dense(n_capa_oculta, activation="relu"),
            keras.layers.Dense(n_salida, activation="softmax")
        ])
else:
    modelo = None
```

```

else:
    if batch_dropout:
        modelo = keras.Sequential([

            keras.layers.Conv2D(filters = n_filtros, kernel_size=tam_filtro, activation = "relu", input_shape=[tamX,tamY,RGB]),
            keras.layers.MaxPool2D(pool_size=tam_pool),
            keras.layers.BatchNormalization(),
            keras.layers.Dropout(dropout),
            keras.layers.Conv2D(filters = int(n_filtros/2), kernel_size=int(tam_filtro/1.5), activation = "relu"),
            keras.layers.MaxPool2D(pool_size=tam_pool),
            keras.layers.BatchNormalization(),
            keras.layers.Flatten(),
            keras.layers.Dropout(dropout),

            keras.layers.Dense(n_capa_oculta, activation="relu"),
            keras.layers.Dense(n_salida, activation="softmax")

        ])
    else:
        modelo = keras.Sequential([

            keras.layers.Conv2D(filters = n_filtros, kernel_size=tam_filtro, activation = "relu", input_shape=[tamX,tamY,RGB]),
            keras.layers.MaxPool2D(pool_size=tam_pool),
            keras.layers.Conv2D(filters = int(n_filtros/2), kernel_size=int(tam_filtro/1.5), activation = "relu"),
            keras.layers.MaxPool2D(pool_size=tam_pool),
            keras.layers.Flatten(),

            keras.layers.Dense(n_capa_oculta, activation="relu"),
            keras.layers.Dense(n_salida, activation="softmax")

        ])

```

Ilustración 4.2: Creación del Modelo de la CNN

Finalmente se compilará y entrenará el modelo exactamente igual que en la práctica anterior debido a que tras realizar el proceso de convolución, se obtiene un vector (capa “Flatten”) el cual se introduce a un Perceptrón Multicapa.

Como se trata de un problema de clasificación, se elegirá una función de activación “softmax” y una función de pérdida “categorical_crossentropy” o “sparse_categorical_crossentropy” (dependiendo de si elegimos “one_hot_encoding” o no).

c. RESULTADOS

Con el fin de mostrar el funcionamiento de las Red Neuronal Convolutacional entrenada, se utilizará la función “Info_CNN” las cual grafica los resultados, reduce el número de dimensiones, elimina el escalado, realiza la matriz de confusión...

```
def Info_CNN (dataset = None,
              X_train = None, Y_train = None, X_val = None, Y_val = None, X_test = None, Y_test = None,
              scaler = False,
              resultados = None, matriz_conf = False, datos_terminal = False,
              Y_pred = None, one_hot_encoding = False):
```

Ilustración 4.3: Funcion “Info_CNN”

A continuación se comentarán los 2 ejercicios deseados.

- Ejercicio 1 – Conjunto de datos Fashion MNIST.

```
fashion_mnist = keras.datasets.fashion_mnist
(X_train, Y_train), (X_test, Y_test) = fashion_mnist.load_data()
modelo, resultados, X_train, Y_train, X_val, Y_val, X_test, Y_test = CNN(X_train = X_train, Y_train = Y_train, X_val = X_test, Y_val = Y_test, X_test = X_test, Y_test = Y_test,
                              epocas = 30, scaler = True, one_hot_encoding = True)
Y_pred = modelo.predict(X_test)
Info_CNN( X_train = X_train, Y_train = Y_train, X_val = X_test, Y_val = Y_test, X_test = X_test, Y_test = Y_test,
          scaler = True,
          resultados = resultados, matriz_conf = True, datos_terminal = True,
          Y_pred = Y_pred, one_hot_encoding = True)
```

Ilustración 4.4: Ejecución del Ejercicio 1.

Tras 30 épocas de entrenamiento se obtiene la siguiente gráfica de evolución mostrando como la precisión aumenta mientras que la pérdida disminuye por cada época que pasa. Se puede observar como la pérdida de los datos validados asciende pasada la época 4 y la precisión de los datos de validación no aumentan. Si se hubiese utilizado “Parada Temprana” se podría haber solucionado.

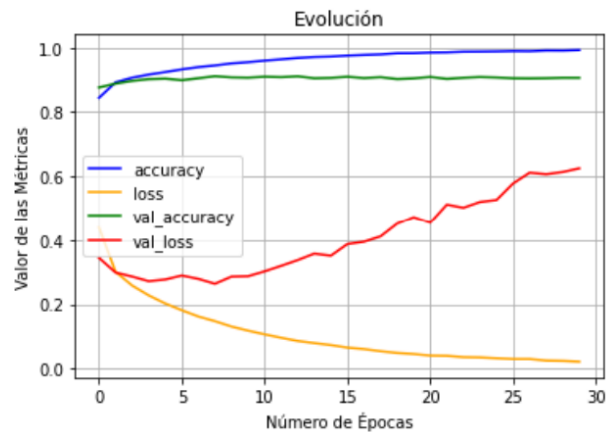


Ilustración 4.5: Evolución Ejercicio 1

Igualmente observando la matriz de confusión se comprueba cómo los datos obtenidos son bastante buenos sin existir prácticamente ningún error.

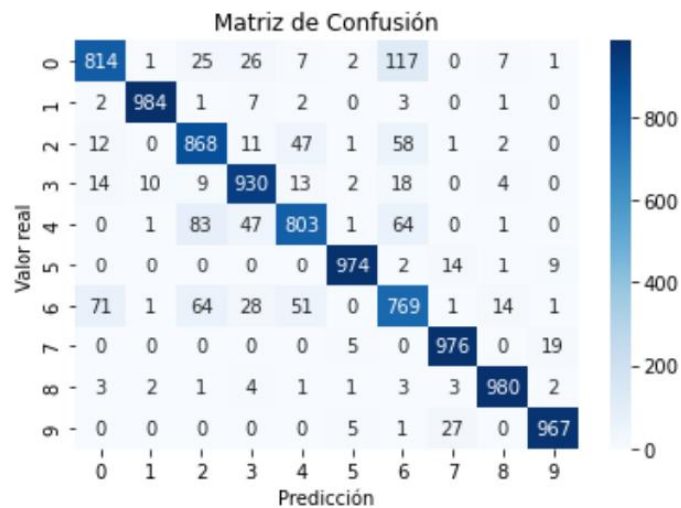


Ilustración 4.6: Matriz de Confusión Ejercicio 1

- Ejercicio 2 – Conjunto de datos CIFAR10.

```
(X_train, Y_train), (X_test, Y_test) = keras.datasets.cifar10.load_data()
modelo, resultados, X_train, Y_train, X_val, Y_val, X_test, Y_test = CNN(X_train = X_train, Y_train = Y_train, X_val = X_test, Y_val = Y_test, X_test = X_test, Y_test = Y_test,
epocas = 30, scaler = True, one_hot_encoding = True, n_capas_CNN = 2, batch_dropout = False)

Y_pred = modelo.predict(X_test)

Info_CNN(X_train = X_train, Y_train = Y_train, X_val = X_test, Y_val = Y_test, X_test = X_test, Y_test = Y_test,
scaler = True,
resultados = resultados, matriz_conf = True, datos_terminal = True,
Y_pred = Y_pred, one_hot_encoding = True)
```

Ilustración 4.7: Ejecución del Ejercicio 2.

Tras 30 épocas de entrenamiento se obtiene la siguiente gráfica de evolución mostrando como la precisión aumenta mientras que la pérdida disminuye por cada época que pasa.

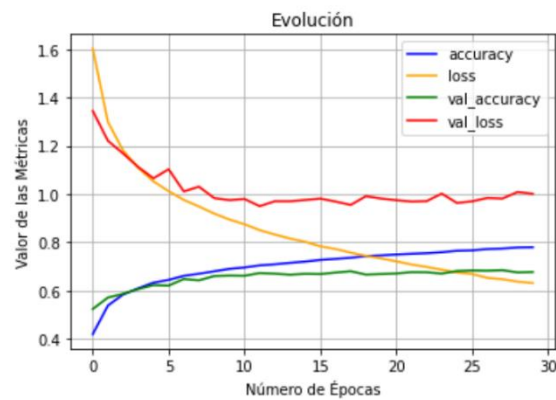


Ilustración 4.8: Evolución Ejercicio 2

Observando la matriz de confusión se comprueba cómo los datos obtenidos son bastante buenos sin existir prácticamente ningún error.

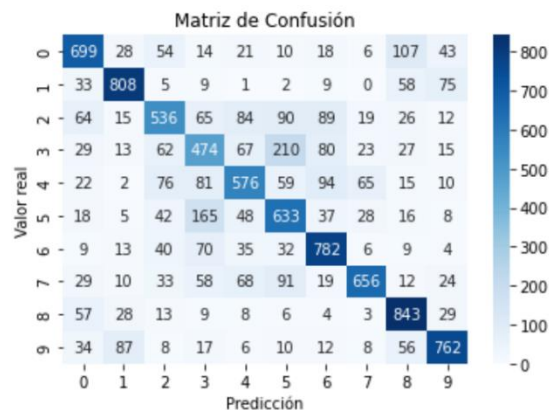


Ilustración 4.9: Matriz de Confusión Ejercicio 2

5. TRATAMIENTO DE DATOS

a. OBJETIVOS

Dado un excel con datos, se pide tratarlo (eliminando datos repetidos, nulos...) con el fin de entrenar una red neuronal para predecir si una persona con ciertas características cobra o no más de 50000€ al año. Para ello se ha creado un código interactivo en el que el usuario deberá utilizar la terminal para decidir que datos mantiene, cuales cambia y cuales elimina.

b. DATOS NULOS Y DUPLICADOS, Y ELIMINACIÓN DE COLUMNAS.

Se han creado 2 funciones nuevas con el fin de buscar y eliminar datos nulos o repetidos.

La primera función es llamada “tratar_nan_null_none()” y se encarga de lo siguiente:

- Mostrar por pantalla el número de datos NaN existentes en el dataset.

```
print()
print("Suma de valores NaN por columna:")
print(data.isna().sum())

input()

print()
print("Porcentaje de valores NaN por columna (%):")
print(data.isna().mean() * 100)
```

Ilustración 5.1: Mostrar Cantidad de NaN

En el caso de este dataset, no existen datos nulos.

```
Suma de valores NaN por columna:
age                0
workclass          0
fnlwgt             0
education          0
education-num      0
marital-status     0
occupation         0
relationship       0
race               0
sex                0
capital-gain       0
capital-loss       0
hours-per-week     0
native-country     0
income             0
dtype: int64
```

Ilustración 5.2: Cantidad de NaN en Dataset

- Borrar alguna columna si fuese necesario

```
col = data.columns[borrar_colum]
data.drop(columns=[col], inplace = True)
print()
print("Columna borrada:", col )
print()
```

Ilustración 5.3: Borrar Columnas

- Sustituir los valores NaN de una columna por su media, mediana o moda.

```
med_median_mod = input("Sustituir NaN por 'media' (med), 'mediana' (median) o 'moda' (mod): ")

if med_median_mod == "med":
    data[col].fillna(data[col].mean(), inplace=True)
elif med_median_mod == "median":
    data[col].fillna(data[col].median(), inplace=True)
elif med_median_mod == "mod":
    data[col].fillna(data[col].mode()[0], inplace=True)
```

Ilustración 5.4: Sustituir valores NaN

- Eliminar filas con valores NaN

```
antes = len(data)
data.dropna(inplace=True)
despues = len(data)
print()
print("Filas eliminadas: " , antes - despues , "de" , antes , "a" , despues)
print()
```

Ilustración 5.5: Eliminar Filas

Al no existir valores NaN, no se empleará ninguno de los métodos que acabamos de ver.

Por otro lado se utilizará la función “tratar_duplicados()” la cual muestra el número de datos duplicados y los elimina si el usuario lo pide.

```
def tratar_duplicados(data):

    print()
    print(f'Registros duplicados: {data.duplicated().sum()}')
    print(f'Tamaño del set antes de eliminar las filas repetidas: {data.shape}')

    borrar_fila = input("¿Quieres borrar filas duplicadas? (s/S/y/Y): ")

    if borrar_fila in {"s", "S", "y", "Y"}:
        data.drop_duplicates(inplace=True)
        print(f'Tamaño del set después de eliminar las filas repetidas: {data.shape}')

    print()
    return data
```

Ilustración 5.6: Función “tratar_duplicados()”

Usando la función se puede observar como hay 24 registros duplicados, los cuales se eliminarán.

c. TRATAR TEXTOS

Para tratar las columnas con datos “textuales” y no numéricos se ha creado una función llamada “tratar_textos()” la cual realiza lo siguiente:

- Poner los nombres de los datos en minúscula.

```
minuscula = input("¿Quieres poner los nombres en minúscula? (s/S/y/Y): ")
if minuscula in {"s", "S", "y", "Y"}:
    data[col] = data[col].str.lower()

    print()
    sns.countplot(x=col, data=data)
    plt.xticks(rotation=270)
    plt.show()
    print(data[col].unique())
    print()
```

Ilustración 5.7: Poner datos en Minúscula.

- Unificar dos tipos de datos en uno solo.

```
unificar = input("¿Quieres unificar columnas? (s/S/y/Y): ")
if unificar in {"s", "S", "y", "Y"}:
    a = input("¿Qué columna quieres mantener? (escribe el texto): ")
    b = input("¿Qué columna quieres escribir en "+ a +"? (escribe el texto): ")

    if b in a:
        data.loc[data[col]==b,col] = a
    else:
        data[col] = data[col].str.replace(b,a, regex=False)

    print()
    sns.countplot(x=col, data=data)
    plt.xticks(rotation=270)
    plt.show()
    print(data[col].unique())
    print()
```

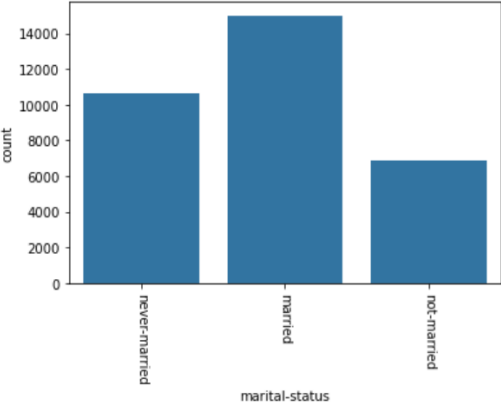
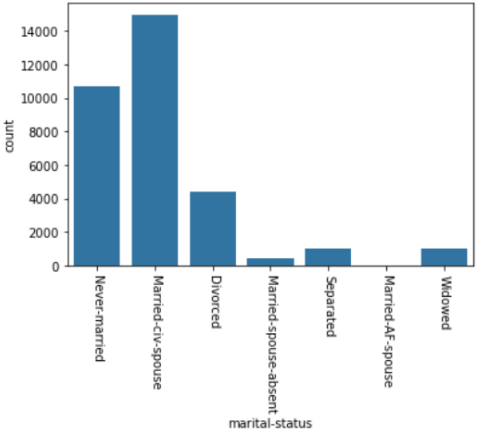
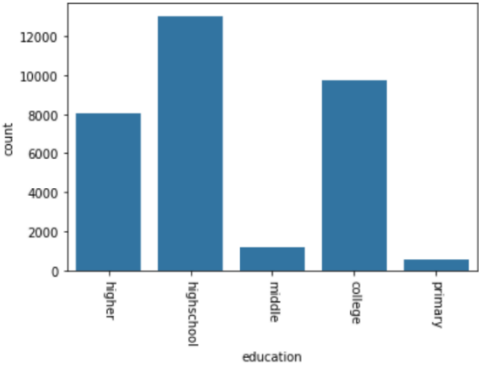
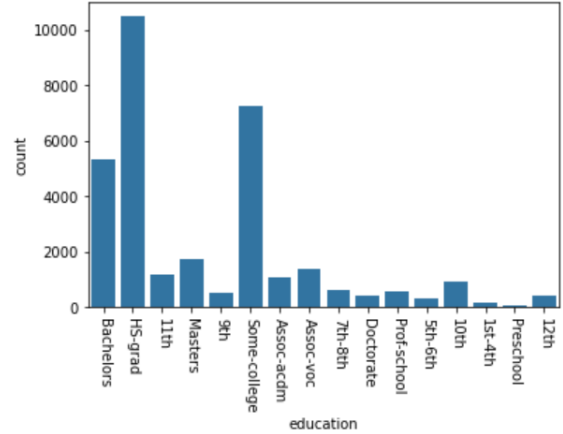
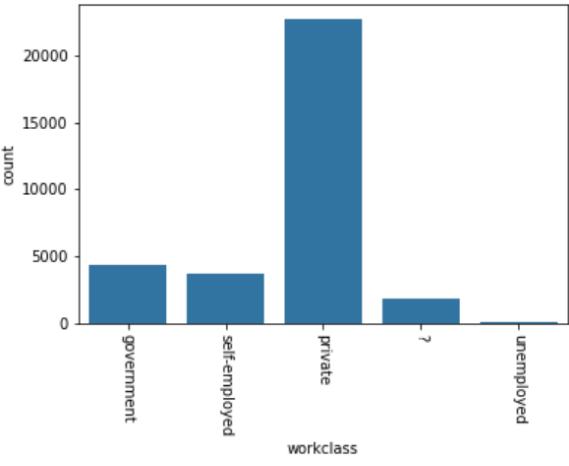
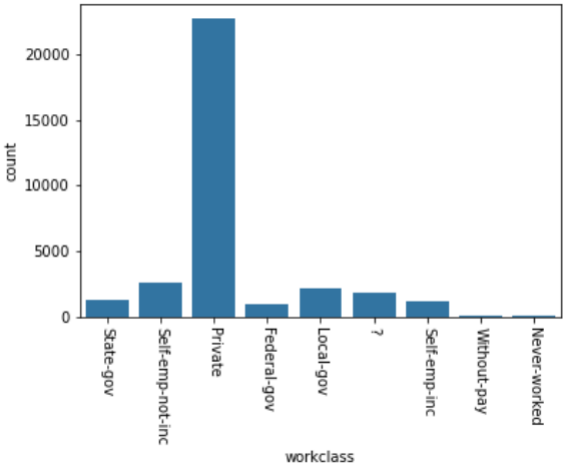
Ilustración 5.8: Unificar Datos

- Convertir todos los datos del texto en una o más columnas de tipo “Dummy”

```
dummy = input("¿Convertir Columna en Dummy? (s/S/y/Y): ")
if dummy in {"s", "S", "y", "Y"}:
    data[col] = data[col].astype(str).str.strip().str.capitalize()
    data = pd.get_dummies(data, columns=[col], drop_first=True, dtype = int)
```

Ilustración 5.9: Convertir en Dummy

Finalmente, para el dataset querido se harán estas unificaciones antes de convertirse en datos “Dummy”



d. TRATAR OUTLIERS

Con el fin de tratar los datos atípicos o irreales se utilizará la función “tratar_outliers()” la cual mantiene los datos que se encuentren entre 2 números.

```
borrar_outliers = input("¿Quieres borrar Outliers? (s/S/y/Y): ")
if borrar_outliers in {"s", "S", "y", "Y"}:
    a = int(input("Introduce el número MÍNIMO a mantener: "))
    b = int(input("Introduce el número MÁXIMO a mantener: "))

    print(f'Tamaño del set antes de eliminar registros de edad: {data.shape}')

    data = data[data[col]<=b]
    data = data[data[col]>a]

    print(f'Tamaño del set después de eliminar registros de edad: {data.shape}')

    sns.boxplot(x=col, data=data)
    plt.show()
```

Ilustración 5.12: Función “tratar_outliers()”.

Sin embargo, al observar las variables numéricas que nos quedan, aunque existan datos poco probables, todos los datos son posibles, por lo que no se eliminará ningún dato.

e. PERCEPTRÓN MULTICAPA

Finalmente se utiliza el nuevo “dataset” en las funciones creadas en la práctica 3 con el fin de predecir si una persona es capaz de cobrar más o menos de 50000 dólares.

Para ello se separa la información en datos “X” (atributos) e “Y” (resultado).

```
Y = data["income_>50k"]  
X = data.drop(columns = ["income_>50k"])
```

Ilustración 5.13: Separación de Datos

Para ello se modificarán las funciones “Clasificacion_Multi()” y “Info_Clas_Mult()” de la práctica 3 para el tipo de problema de clasificación binaria. Para ello se utilizará una función de activación “sigmoid” y una función de pérdida de “binary_crossentropy”. Las funciones se utilizarán de la siguiente manera:

```
X_test_val, X_train, Y_test_val, Y_train = train_test_split(  
    X, Y, test_size=0.7, shuffle=True)  
  
X_val, X_test, Y_val, Y_test = train_test_split(  
    X_test_val, Y_test_val, test_size=0.5, shuffle=True)  
  
# No sé con cual está mejor, si con "one_hot" o sin.  
modelo, resultados, X_train, Y_train, X_val, Y_val, X_test, Y_test, scaler = Clasificacion_Multi(X_train = X_train, Y_train = Y_train, X_val = X_val, Y_val = Y_val, X_test = X_test, Y_test = Y_test,  
    epocas = 200, one_hot_encoding = False)  
  
Y_pred = modelo.predict(X_test)  
  
Info_Clas_Mult( X_train = X_train, Y_train = Y_train, X_val = X_val, Y_val = Y_val, X_test = X_test, Y_test = Y_test,  
    scaler = scaler,  
    resultados = resultados, matriz_conf = True, datos_terminal = True,  
    Y_pred = Y_pred, one_hot_encoding = False)
```

Ilustración 5.14: Uso del Perceptrón Multicapa.

A continuación se observa cómo la matriz de confusión muestra que la mayoría de veces acierta la predicción, pero otras muchas no lo consigue.

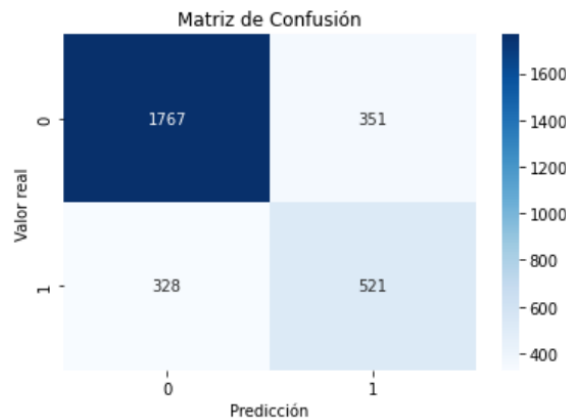


Ilustración 5.15: Matriz de Confusión.